Chapter 5. Boosting Methods

Predictive Modeling, Lecture 4 (AdaBoost, gradient boosting, XGBoost, LightGBM, CatBoost)

Contents

1 From Averaging to Sequential Correction **2**

2 AdaBoost **2**

2.1 Algorithm 2

2.2 AdaBoost is forward stagewise fitting of the exponential loss 2

3 Gradient Boosting Machines **3**

3.1 Algorithm 3

3.2 Loss functions and their pseudo-residuals 4

3.3 Shrinkage, subsampling and early stopping 4

3.4 Limitations of the standard GBM 4

4 XGBoost: Regularized Second-Order Boosting **4**

5 LightGBM **5**

6 CatBoost **6**

6.1 The curse of dimensionality of categorical variables 6

6.2 Naive target statistics leak the target 6

7 Summary and Practical Rules **6**

8 Selected exercises **7**

1 From Averaging to Sequential Correction

Random forest (Ch. 4)	Gradient boosting (this chapter)
$\hat{f}_{\mathrm{RF}}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$	$F_M(x) = F_0(x) + \sum_{m=1}^{M} \eta\, h_m(x)$
trees learned **independently** on different bootstrap samples	h_m: a shallow tree learning the **error (residual) direction** left by the previous model
averaging reduces the instability of individual trees	η: the **learning rate**, shrinking each correction to prevent overfitting
every tree looks at the same target Y	small corrections ηh_m accumulate **sequentially**, not independently

Random forests mainly reduce variance; boosting mainly reduces bias with many shallow learners.

2 AdaBoost

Motivation. AdaBoost reweights toward current errors, which minimizes the exponential loss stagewise.

2.1 Algorithm

Algorithm: AdaBoost (labels $y_i \in \{-1, +1\}$)

Initialize $D_1(i) = 1/N$. For $b = 1, \ldots, B$:

1. **Train**: fit a classifier $h_b(x) \in \{-1, +1\}$ using weights D_b.
2. **Evaluate**: weighted error $\epsilon_b = \sum_{i=1}^{N} D_b(i)\, \mathbb{I}\{h_b(x_i) \neq y_i\}$.
3. **Weight**: $\alpha_b = \frac{1}{2} \log \dfrac{1-\epsilon_b}{\epsilon_b}$.
4. **Update**: $D_{b+1}(i) = \dfrac{D_b(i) \exp\{-\alpha_b y_i h_b(x_i)\}}{Z_b}$, Z_b the normalizer.

Output: $H(x) = \mathrm{sign}\Big(\sum_{b=1}^{B} \alpha_b h_b(x)\Big)$.

Errors are multiplied by e^{α_b}, correct cases by $e^{-\alpha_b}$; smaller ϵ_b gives larger α_b ($\epsilon_b = \frac{1}{2} \Rightarrow \alpha_b = 0$; $\epsilon_b > \frac{1}{2}$ flips the learner).

2.2 AdaBoost is forward stagewise fitting of the exponential loss

Theorem: derivation of α_b and the weight update

Let $F_{b-1} = \sum_{s<b} \alpha_s h_s$ and consider the exponential loss $L(y, F) = e^{-yF}$. Choosing (α, h) to minimize $\sum_i \exp\{-y_i(F_{b-1}(x_i) + \alpha h(x_i))\}$ gives: h_b minimizes the weighted error with weights $D_b(i) \propto e^{-y_i F_{b-1}(x_i)}$, and $\alpha_b = \frac{1}{2} \log \frac{1-\epsilon_b}{\epsilon_b}$. The weights evolve exactly by the AdaBoost update.

Proof [Supplement]. With $w_i = e^{-y_i F_{b-1}(x_i)}$, normalized to $D_b(i)$, and $y_i h(x_i) = \pm 1$,

$$\sum_i D_b(i) e^{-\alpha y_i h(x_i)} = e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon = e^{-\alpha} + (e^{\alpha} - e^{-\alpha})\epsilon.$$

For $\alpha > 0$ this increases in ϵ, so h_b minimizes weighted error; ∂_α: $-e^{-\alpha}(1-\epsilon)+e^{\alpha}\epsilon = 0 \iff e^{2\alpha} = (1-\epsilon)/\epsilon$; and $w_i^{\text{new}} = w_i e^{-\alpha_b y_i h_b(x_i)}$ normalizes to the update. ■

Theorem: training-error bound

$Z_b = 2\sqrt{\epsilon_b(1-\epsilon_b)}$ and

$$\frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\{H(x_i)\neq y_i\} \le \prod_{b=1}^{B} Z_b \le \exp\Big(-2\sum_{b=1}^{B}\gamma_b^2\Big), \qquad \gamma_b = \tfrac{1}{2}-\epsilon_b.$$

Proof [Supplement]. $Z_b = e^{-\alpha_b}(1-\epsilon_b)+e^{\alpha_b}\epsilon_b = 2\sqrt{\epsilon_b(1-\epsilon_b)}$. Unrolling, $D_{B+1}(i) = \frac{e^{-y_i F_B(x_i)}}{N\prod_b Z_b}$ sums to 1, so $\frac{1}{N}\sum_i e^{-y_i F_B(x_i)} = \prod_b Z_b$; and $\mathbb{I}\{H(x_i)\neq y_i\} \le e^{-y_i F_B(x_i)}$. Finally $2\sqrt{\epsilon(1-\epsilon)} = \sqrt{1-4\gamma^2} \le e^{-2\gamma^2}$. ■

With $\gamma_b \ge \gamma > 0$ training error falls exponentially. Minimizing $pe^{-f}+(1-p)e^{f}$ gives $f^\star(x) = \frac{1}{2}\log\frac{P(Y=1|x)}{P(Y=-1|x)}$: half the log-odds.

Example: AdaBoost on a toy case (classic illustration)

Ten points, decision stumps. Round 1: vertical stump, three + errors, $\epsilon_1 = 0.30$, $\alpha_1 = \frac{1}{2}\log(0.7/0.3) = 0.42$. Round 2: vertical, $\epsilon_2 = 0.21$, $\alpha_2 \approx 0.65$. Round 3: horizontal, $\epsilon_3 = 0.14$, $\alpha_3 \approx 0.92$:

$$H_{\text{final}} = \text{sign}(0.42\,h_1 + 0.65\,h_2 + 0.92\,h_3)$$

is a staircase no single stump represents. (The formula gives 0.66, 0.91; the figures round ϵ_b.)

3 Gradient Boosting Machines

3.1 Algorithm

Algorithm: GBM

Initialize $F_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i,\gamma)$. For $b = 1,\dots,B$:

1. **Gradient**: pseudo-residuals $r_{ib} = -\Big[\dfrac{\partial L(y_i,F(x_i))}{\partial F(x_i)}\Big]_{F=F_{b-1}}$.
2. **Fit**: fit a regression tree $h_b(x)$ (leaves R_{jb}) to the targets r_{ib}; optionally set each leaf value by a line search $\gamma_{jb} = \arg\min_\gamma \sum_{x_i\in R_{jb}} L(y_i, F_{b-1}(x_i)+\gamma)$.
3. **Update**: $F_b(x) = F_{b-1}(x) + \eta\, h_b(x)$.

Output: $F_B(x) = F_0(x) + \sum_{b=1}^{B}\eta\, h_b(x)$.

For squared error $r_{ib} = y_i - F_{b-1}(x_i)$; **any differentiable loss** works. This is **gradient descent in function space**: $(F(x_1),\dots,F(x_N))$ moves along the negative gradient, approximated by a tree that extends to new x.

3.2 Loss functions and their pseudo-residuals

Loss	$\Psi(y,f)$	$-\partial\Psi/\partial f$	F_0
squared (L_2)	$\frac{1}{2}(y-f)^2$	$y-f$	mean
absolute (L_1)	$\lvert y-f\rvert$	$\operatorname{sign}(y-f)$	median
Huber	$\frac{1}{2}(y-f)^2$ if $\lvert y-f\rvert\le\delta$; $\delta(\lvert y-f\rvert-\delta/2)$ otherwise	$y-f$ or $\delta\operatorname{sign}(y-f)$	robust center
Bernoulli	$\log(1+e^{-2\tilde{y}f})$, $\tilde{y}\in\{-1,1\}$	$\frac{2\tilde{y}}{1+e^{2\tilde{y}f}}$	$\frac{1}{2}\log\frac{\bar{p}}{1-\bar{p}}$
exponential (AdaBoost)	$e^{-\tilde{y}f}$	$\tilde{y}e^{-\tilde{y}f}$	$\frac{1}{2}\log\frac{\bar{p}}{1-\bar{p}}$

Bernoulli: f is half the log-odds ($p=1/(1+e^{-2f})$); on the log-odds scale the pseudo-residual is $y-p$, $y\in\{0,1\}$. Absolute/Huber losses limit large residuals (Ch. 2).

3.3 Shrinkage, subsampling and early stopping

Small η (0.01–0.1) needs more trees but generalizes better. **Stochastic gradient boosting** subsamples rows/columns per tree. B is set by **early stopping** on validation loss. η, B, depth (interaction order) and subsampling control the *same* overfitting.

3.4 Limitations of the standard GBM

Standard GBM overfits with too many trees, has no explicit complexity penalty, and is slow.

4 XGBoost: Regularized Second-Order Boosting

Motivation. A second-order expansion gives closed-form leaf values and a regularized split gain.

- **Regularization**: L_1/L_2 penalties on leaf weights and a per-leaf cost.
- **Second-order optimization**: uses the Hessian, not only the gradient.

Theorem: optimal leaf weights and split gain

At round t the new tree $f_t(x)=w_{q(x)}$ (leaf index $q(x)$, T leaves) minimizes

$$\mathcal{L}^{(t)}=\sum_i L(y_i,\hat{y}_i^{(t-1)}+f_t(x_i))+\gamma T+\tfrac{1}{2}\lambda\sum_{j=1}^{T}w_j^2.$$

With $g_i=\partial_{\hat{y}}L(y_i,\hat{y}_i^{(t-1)})$, $h_i=\partial^2_{\hat{y}}L(y_i,\hat{y}_i^{(t-1)})$, $G_j=\sum_{i\in I_j}g_i$, $H_j=\sum_{i\in I_j}h_i$, the second-order approximation gives

$$w_j^\star=-\frac{G_j}{H_j+\lambda},\qquad \tilde{\mathcal{L}}^\star=-\frac{1}{2}\sum_{j=1}^{T}\frac{G_j^2}{H_j+\lambda}+\gamma T,$$

and the gain of splitting a leaf into L and R is

$$\text{Gain}=\frac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda}+\frac{G_R^2}{H_R+\lambda}-\frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\Big]-\gamma.$$

Proof [Supplement]. $L(y_i, \hat{y} + f) \approx L(y_i, \hat{y}) + g_i f + \frac{1}{2} h_i f^2$; grouping by leaves, $\tilde{\mathcal{L}} = \sum_j \{G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2\} + \gamma T$, minimized at $w_j = -G_j/(H_j + \lambda)$ with value $-\frac{1}{2} G_j^2/(H_j + \lambda)$. Gain = one-leaf objective − two-leaf objective $-\gamma$. ■

Example: what $w^\star$ is for familiar losses

Squared loss: $g_i = \hat{y}_i - y_i$, $h_i = 1$, $w_j^\star = \sum_{i \in I_j} (y_i - \hat{y}_i)/(n_j + \lambda)$, a *shrunken* mean residual. Logistic: $g_i = p_i - y_i$, $h_i = p_i(1 - p_i)$, one Newton step. `min_child_weight` bounds H_j below; γ prunes weak splits.

System side: learned default direction for missing values, quantile-sketch candidates, cache-aware parallelism.

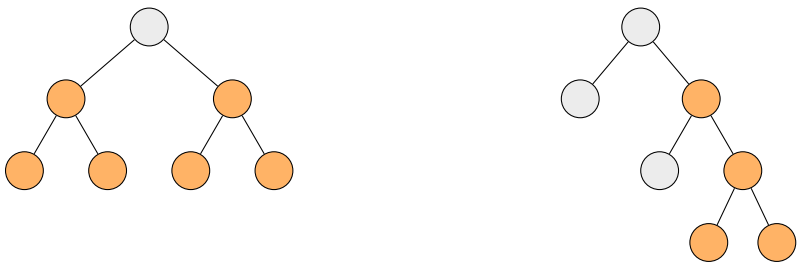
5 LightGBM

- **Histogram-based learning**: continuous variables bucketed into few bins.
- **Leaf-wise tree growth**: expand only the leaf with the largest loss reduction.

Aspect	Standard GBM	LightGBM
split search	pre-sorts every continuous value and scans all of them; bottleneck on large data	integer bins (histogram) cut computation and memory
tree growth	level-wise: grows all nodes of a level symmetrically, even unnecessary ones	leaf-wise: expands only the node that reduces the error most

Derivation: why histograms are cheap

With k bins (default 255), a node's (G, H) histogram costs $O(\#\text{rows})$ per feature and the split search $O(k)$. A child's histogram is parent minus sibling (*histogram subtraction*). Coarse bins mildly regularize.



level-wise (GBM, XGBoost default) leaf-wise (LightGBM)

Figure 1: Orange: most recent splits. Leaf-wise growth gives deep, unbalanced trees for the same number of leaves.

Remark: leaf-wise growth and overfitting

Lower training loss per leaf, but overfits small data: control `num_leaves`, `max_depth`, `min_data_in_leaf`. Also GOSS (keep large $|g_i|$, reweighted subsample of the rest) and EFB (bundle rarely co-nonzero sparse features).

6 CatBoost

Motivation. CatBoost computes everything for a row from earlier rows only, so its own label never leaks.

6.1 The curse of dimensionality of categorical variables

- GBM: one-hot encoding gives hundreds of dummy columns.
- CatBoost: one column of target = 1 proportions (target statistics).

Example: one-hot (GBM) versus target statistics (CatBoost)

Category	Share of target = 1 in that category	CatBoost value (one column)
A	80 of 100 (80%)	$\to 0.80$
B	10 of 40 (25%)	$\to 0.25$
C	100 of 200 (50%)	$\to 0.50$
D	5 of 50 (10%)	$\to 0.10$

One column replaces four dummies.

6.2 Naive target statistics leak the target

Theorem: target leakage of in-sample target encoding

If each row's encoding uses its own target, $\hat{x}_i = \frac{1}{n_{c_i}} \sum_{j:c_j=c_i} y_j$, then for a category observed once the encoded value equals y_i. In the extreme of all categories unique (e.g. an ID), the encoded feature equals the target on the training data and a single split fits it perfectly, while on new data the feature carries no information.

Generally $\mathbb{E}[\hat{x}_i \mid y_i]$ depends on y_i via y_i/n_{c_i}, so training and test encodings differ (*target shift*). CatBoost's fix:

- **smoothing with a prior**: $\hat{x}_i = \dfrac{\sum_{j\in\mathcal{P}_i} \mathbb{I}(c_j = c_i)\, y_j + a\,\bar{p}}{\sum_{j\in\mathcal{P}_i} \mathbb{I}(c_j = c_i) + a}$, shrinking rare categories toward the global rate $\bar{p}$;
- **ordered target statistics**: for a random permutation σ, $\mathcal{P}_i = \{j : \sigma(j) < \sigma(i)\}$, so only *earlier* rows encode row i, as at test time.

Ordered boosting likewise computes a row's residual with a model that has not seen it (no "prediction shift"). Symmetric (oblivious) trees use one split per level: fast and regularizing.

7 Summary and Practical Rules

Algorithm	Direction of development
XGBoost	explicit regularization controls overfitting; second-order information (Hessian) gives precise optimization
LightGBM	histograms and leaf-wise growth drastically reduce computation on large numerical/sparse data
CatBoost	ordered target statistics handle leakage and instability of categorical features

Rules for using boosting.

- Boosting adds shallow trees along the steepest loss decrease.
- Learning rate, iterations, tree complexity, subsampling and early stopping control one overfitting problem.
- Choose by repeated performance under a fixed validation procedure (Ch. 1 rules; early stopping on a validation fold, never the test set).

8 Selected exercises

Motivation. AdaBoost's reset to chance, L_2 boosting as shrinkage, and the XGBoost gain versus CART.

Exercise 5.1 (The last weak learner becomes a coin flip)

In AdaBoost with $0 < \epsilon_b < 1$, show that the weighted error of h_b under the updated weights is exactly one half:

$$\sum_{i=1}^{N} D_{b+1}(i)\, \mathbb{I}\{h_b(x_i) \neq y_i\} = \tfrac{1}{2}.$$

Solution [Supplement].

Step 1. Errors ($y_i h_b = -1$) get $e^{\alpha_b} = \sqrt{(1-\epsilon_b)/\epsilon_b}$, correct cases $e^{-\alpha_b}$: unnormalized totals $\epsilon_b e^{\alpha_b} = \sqrt{\epsilon_b(1-\epsilon_b)} = (1-\epsilon_b)e^{-\alpha_b}$.

Step 2. So $Z_b = 2\sqrt{\epsilon_b(1-\epsilon_b)}$ and the errors' normalized weight is $\frac{1}{2}$. ■

Exercise 5.2 (L_2 boosting with a linear smoother)

Use squared loss, $F_0 = \mathbf{0}$ (centered $\mathbf{y}$), and a base learner that maps a target vector r to the fitted vector Sr, where S is a symmetric $N \times N$ matrix with eigenvalues $\mu_1, \ldots, \mu_N \in [0,1]$ (e.g. the OLS hat matrix, or a Ridge smoother). The fitted vector is updated by $F_b = F_{b-1} + \nu S(\mathbf{y} - F_{b-1})$ with $0 < \nu \leq 1$.
(a) Show that $F_B = \{I - (I - \nu S)^B\}\mathbf{y}$.
(b) With $S = \sum_k \mu_k u_k u_k^\top$ (orthonormal u_k), show that $F_B = \sum_k \{1 - (1-\nu\mu_k)^B\}(u_k^\top \mathbf{y})\, u_k$, that each factor lies in $[0,1]$ and increases to $\mathbb{I}\{\mu_k > 0\}$ as $B \to \infty$, and that the degrees of freedom $\mathrm{tr}\{I - (I - \nu S)^B\}$ increase in B.
(c) For $S = H$ a projection, show that $F_B = \{1 - (1-\nu)^B\}H\mathbf{y}$.

Solution [Supplement].

(a) $R_b = \mathbf{y} - F_b = (I - \nu S)R_{b-1}$, so $R_B = (I - \nu S)^B \mathbf{y}$ and $F_B = \mathbf{y} - R_B$.

(b)

Step 1. $(I - \nu S)^B = \sum_k (1-\nu\mu_k)^B u_k u_k^\top$ and $I = \sum_k u_k u_k^\top$; apply (a).

Step 2. $0 \leq 1 - \nu\mu_k \leq 1$, < 1 iff $\mu_k > 0$: $(1-\nu\mu_k)^B$ is non-increasing, $\to 0$ if $\mu_k > 0$, $\equiv 1$ if $\mu_k = 0$.

Step 3. $\mathrm{tr}\{I - (I - \nu S)^B\} = \sum_k \{1 - (1-\nu\mu_k)^B\}$ is a sum of non-decreasing terms.

(c) Eigenvalues $0, 1$: factor $1 - (1-\nu)^B$ on the range of H, 0 off it.

B acts like $1/\lambda$ in Ridge; real boosting gains come from base learners that adapt to the residuals. ■

Exercise 5.3 (The XGBoost gain and the CART criterion)

(a) For $L = \frac{1}{2}(y - \hat{y})^2$ with current residuals $r_i = y_i - \hat{y}_i^{(t-1)}$, show that $w_j^\star = n_j \bar{r}_j/(n_j + \lambda)$ and that for $\lambda = \gamma = 0$ the gain equals $\frac{1}{2}\Delta_{\text{RSS}}$ of Ch. 4 computed on the residuals.
(b) Show that for $\lambda = \gamma = 0$ and $h_i > 0$ the gain is never negative, but that for $\lambda > 0$ splitting a leaf into two halves with equal (G, H) has negative gain.

Solution [Supplement].

(a)

Step 1. $g_i = -r_i$, $h_i = 1$: $G_j = -n_j \bar{r}_j$, $H_j = n_j$, $w_j^\star = n_j \bar{r}_j/(n_j + \lambda)$.

Step 2. $\lambda = 0$: $G_j^2/H_j = n_j \bar{r}_j^2$ and $n\bar{r} = n_L \bar{r}_L + n_R \bar{r}_R$, so

$$\text{Gain} = \tfrac{1}{2}(n_L \bar{r}_L^2 + n_R \bar{r}_R^2 - n\bar{r}^2) = \tfrac{1}{2}\{n_L(\bar{r}_L - \bar{r})^2 + n_R(\bar{r}_R - \bar{r})^2\},$$

which is half the ANOVA form of Δ_{RSS} for the targets r_i.

(b)

Step 3. By Cauchy–Schwarz,

$$(G_L + G_R)^2 = \Big(\sqrt{H_L}\,\frac{G_L}{\sqrt{H_L}} + \sqrt{H_R}\,\frac{G_R}{\sqrt{H_R}}\Big)^2 \le (H_L + H_R)\Big(\frac{G_L^2}{H_L} + \frac{G_R^2}{H_R}\Big),$$

which is the claim Gain ≥ 0.

Step 4. With $G_L = G_R = g \neq 0$, $H_L = H_R = h$ and $\gamma = 0$,

$$\text{Gain} = \frac{1}{2}\Big[\frac{2g^2}{h + \lambda} - \frac{4g^2}{2h + \lambda}\Big] = -\frac{\lambda g^2}{(h + \lambda)(2h + \lambda)} < 0.$$

Each half has half the curvature and is shrunk more, so λ alone discourages uninformative splits. ■